Settimana 3 - guida illustrata

Calcolo Scientifico in MATLAB. Edizione aggiornata con i grafici di test_multiple e il commento dei risultati. Metodi studiati: bisezione, Newton e secanti; applicazioni: pigreco, boa e radici multiple.

1. Il problema comune

Cerchiamo una posizione alpha tale che f(alpha) = 0. f e una funzione; a, b, x0 e x1 sono numeri scalari, cioe posizioni sull'asse x. f(x0) e invece l'altezza del grafico in quella posizione. Una funzione puo avere piu radici.

2. Tre quantita da non confondere

Errore sulla radice: abs(z-alpha). Misura la distanza dalla soluzione esatta; in generale alpha non e nota. Nei test usiamo valori di riferimento come sqrt(2), 1-pi e 1.

Residuo: res = f(z). Misura quanto l'equazione e soddisfatta, non la distanza dalla radice. Per funzioni molto piatte un residuo minuscolo puo accompagnarsi a un errore di posizione maggiore.

Incremento: abs(xnew-xold). E lo spostamento tra due approssimazioni consecutive. Il vettore ERR dei nostri Newton e secanti contiene questi incrementi, non gli errori esatti.

3. Tolleranza e arresto

tol e la soglia scelta dall'utente. Nella bisezione si applica alla meta della larghezza dell'intervallo; negli altri due codici all'incremento. k conta i passi, kmax e il massimo consentito: raggiungerlo non dimostra convergenza.

4. Come usare questa guida

Prima capisci quale informazione usa il metodo e come aggiorna le variabili. Poi leggi il codice e infine confronta i risultati. I codici sono didattici: assumono input validi e non gestiscono completamente divisioni per zero, stagnazione, NaN e Inf.

Bisezione - restringere un intervallo

Ipotesi e scelta della meta

Servono a < b, f continua su [a,b] e valori non nulli agli estremi di segno opposto. Questo garantisce almeno una radice, non necessariamente una sola. Se un estremo e gia una radice, puoi restituirlo subito.

Calcola c = (a+b)/2. Se f(a) e f(c) sono discordi, conserva [a,c] e assegna b = c; altrimenti, se c non e gia una radice, conserva [c,b] e assegna a = c. Il criterio e il cambio di segno tra due estremi, non il segno di un punto preso da solo.

Che cosa rappresenta err?

La radice resta nell'intervallo: la sua distanza dal centro non puo superare (b-a)/2. Quindi err e un limite superiore all'errore. Dopo k dimezzamenti il limite e L0/2^(k+1), con L0 larghezza iniziale, in aritmetica esatta. Il codice si arresta quando err < tol, salvo arresto anticipato o limite di passi.

```
function [z,k,res] = bisezione(f,a,b,tol,kmax)
    c = (a+b)/2;
    err = (b-a)/2;
    k = 0;
    while err >= tol && k < kmax
        if f(c) == 0
            break;
        end
        if f(a)*f(c) < 0
            b = c;
        else
            a = c;
        end
        c = (a+b)/2;
        err = (b-a)/2;
        k = k+1;
    end
    z = c;
    res = f(z);
end
```

while ripete finche entrambe le condizioni unite da && sono vere. Prima aggiorni un estremo, poi il centro, poi il limite sull'errore. Il codice presume un intervallo valido e non gestisce gli estremi gia radici. I prodotti per confrontare i segni possono essere delicati in casi di underflow/overflow.

Tuo test su x^2-2, [1,2], tol = 1e-8: 26 dimezzamenti, z = 1.4142135605216026, errore rispetto a sqrt(2) circa 1.85e-9. Il metodo non ha usato sqrt(2) nel criterio d'arresto.

Newton - lo zero della tangente

Idea geometrica e formula

Nel punto corrente x0 approssimi f con la tangente: y = f(x0) + df(x0)*(x-x0). Ponendo y = 0 ottieni la prossima approssimazione. Poi ricostruisci la tangente nel nuovo punto.

```
xnew = x0 - f(x0)/df(x0);
```

Input: f e df sono function handle, x0 e un punto iniziale. Non passare f(x0) e df(x0) al posto delle funzioni: passeresti numeri, che MATLAB potrebbe interpretare come array da indicizzare.

```
function [z,k,res,ERR] = newton(f,df,x0,tol,kmax)
    k = 0;
    err = tol+1;
    ERR = [];
    while err >= tol && k < kmax
        xnew = x0 - f(x0)/df(x0);
        err = abs(xnew-x0);
        ERR = [ERR; err];
        k = k+1;
        x0 = xnew;
    end
    z = x0;
    res = f(z);
end
```

Ordine degli aggiornamenti

Prima calcola l'incremento, poi aggiorna x0. Se assegni x0 = xnew prima del confronto, err diventa zero e il ciclo si ferma subito. Se invece non aggiorni mai x0, ripeti sempre lo stesso passo. ERR = [ERR;err] aggiunge un incremento in fondo al vettore colonna.

Convergenza e limiti

Vicino a una radice semplice, con sufficiente regolarita e partenza adeguata, Newton ha convergenza almeno quadratica. Non e una garanzia da qualunque punto. Una derivata nulla impedisce il passo; una molto piccola puo causare salti enormi. Il metodo puo uscire dall'intervallo desiderato o convergere a un'altra radice. Un incremento piccolo non e un limite garantito sull'errore come nella bisezione.

```
f = @(x) x.^2-2;
df = @(x) 2*x;
[z,k,res,ERR] = newton(f,df,1.5,1e-8,100);
```

Secanti - due punti, senza derivata

Idea e formula

La secante passa per (x0,f(x0)) e (x1,f(x1)). Il rapporto tra variazione verticale e orizzontale ne determina la pendenza r. L'incontro della retta con l'asse x fornisce xnew. Non serve la derivata analitica.

```
r = (f(x1)-f(x0))/(x1-x0);
xnew = x1 - f(x1)/r;
```

x0 e il punto precedente e x1 quello corrente. Non sono estremi che dobbiamo mantenere di segno opposto: il metodo standard conserva sempre le due approssimazioni piu recenti.

```
function [z,k,res,ERR] = secanti(f,x0,x1,tol,kmax)
    k = 0;
    err = tol+1;
    ERR = [];
    while err >= tol && k < kmax
        r = (f(x1)-f(x0))/(x1-x0);
        xnew = x1 - f(x1)/r;
        err = abs(xnew-x1);
        ERR = [ERR; err];
        k = k+1;
        x0 = x1;
        x1 = xnew;
    end
    z = x1;
    res = f(z);
end
```

I due errori da evitare

L'incremento si misura rispetto a x1, non a x0. Alla fine del passo assegna prima x0 = x1 e poi x1 = xnew. Se x0 resta fisso non stai usando il metodo standard delle secanti; se inverti le assegnazioni perdi il vecchio punto corrente.

Convergenza e rischi

Sotto le usuali ipotesi locali, per radici semplici l'ordine e circa 1.618, quindi superlineare. Non e garantita la convergenza globale. Se x1 = x0 o f(x1) = f(x0), le formule incontrano una divisione per zero; differenze molto piccole possono rendere il passo numericamente delicato.

```
f = @(x) x.^2-2;
[z,k,res,ERR] = secanti(f,1,2,1e-8,100);
```

esboa - cosa hanno mostrato i tuoi test

```
R = 0.055; rhob = 0.6;
f = @(x) x.^3 - 3*R*x.^2 + 4*R^3*rhob;
df = @(x) 3*x.^2 - 6*R*x;
a = 0; b = 2*R;
```

Nell'intervallo [0,0.11] f cambia segno ed e strettamente decrescente: la radice e unica in quella regione. Il testo fisico non e stato condiviso, quindi non assegniamo a x un significato o unita non verificati.

Risultati osservati

Bisezione, tol = 1e-12: 36 dimezzamenti, z = 0.0623775815135014, residuo circa 2.21e-15.

Newton da 0.05, tol = 1e-8: 3 passi, z = 0.0623775815137494. Newton da 0.01: 6 passi, z = 0.1463595046947323, fuori dall'intervallo richiesto pur con residuo minuscolo.

Secanti da 0.01 e 0.02, tol = 1e-8: 7 passi, z = 0.0623775815137495. Differenza dalla bisezione circa 2.48e-13.

Come interpretare il confronto

Un residuo piccolo non identifica la radice pertinente al problema. Newton puo lasciare la regione assegnata; la bisezione conserva l'intervallo. Non confrontare solo i passi: criteri d'arresto, tolleranze e costo delle singole iterazioni differiscono.

Usa nomi separati per i risultati: z_bis, z_new, z_sec. abs(z_new-z_bis) misura una differenza tra approssimazioni, non l'errore esatto. Evita di salvare il riferimento dopo che e stato sovrascritto.

Pigreco - collegamento agli arrotondamenti

```
f = zeros(N,1); f(2) = 2;
for n = 2:N-1
    f(n+1) = 2^(n-0.5) ...
        *sqrt(1-sqrt(1-4^(1-n)*f(n)^2));
end
e = abs(f-pi)/pi;
semilogy(2:N,e(2:N),'o');
```

Hai osservato un minimo dell'errore intorno a n = 16 e poi un peggioramento. La sottrazione 1-sqrt(1-delta) e delicata quando delta e piccolo. Se il calcolo produce zero, la successione rimane a zero e l'errore relativo vale 1. Piu iterazioni non significa necessariamente piu accuratezza.

test_multiple - capire le due radici

I due file originali usano la stessa funzione: esmultiple.m applica Newton, esmultiple2.m applica le secanti. test_multiple riunisce i quattro esperimenti e li confronta.

```
f = @(x) sin(x-1) - 0.5*sin(2*(x-1));
df = @(x) cos(x-1) - cos(2*(x-1));
```

Ponendo u = x-1, l'identita sin(2u) = 2*sin(u)*cos(u) porta a f(x) = sin(u)*(1-cos(u)). Le due radici nell'intervallo [-3,3] sono 1-pi e 1.

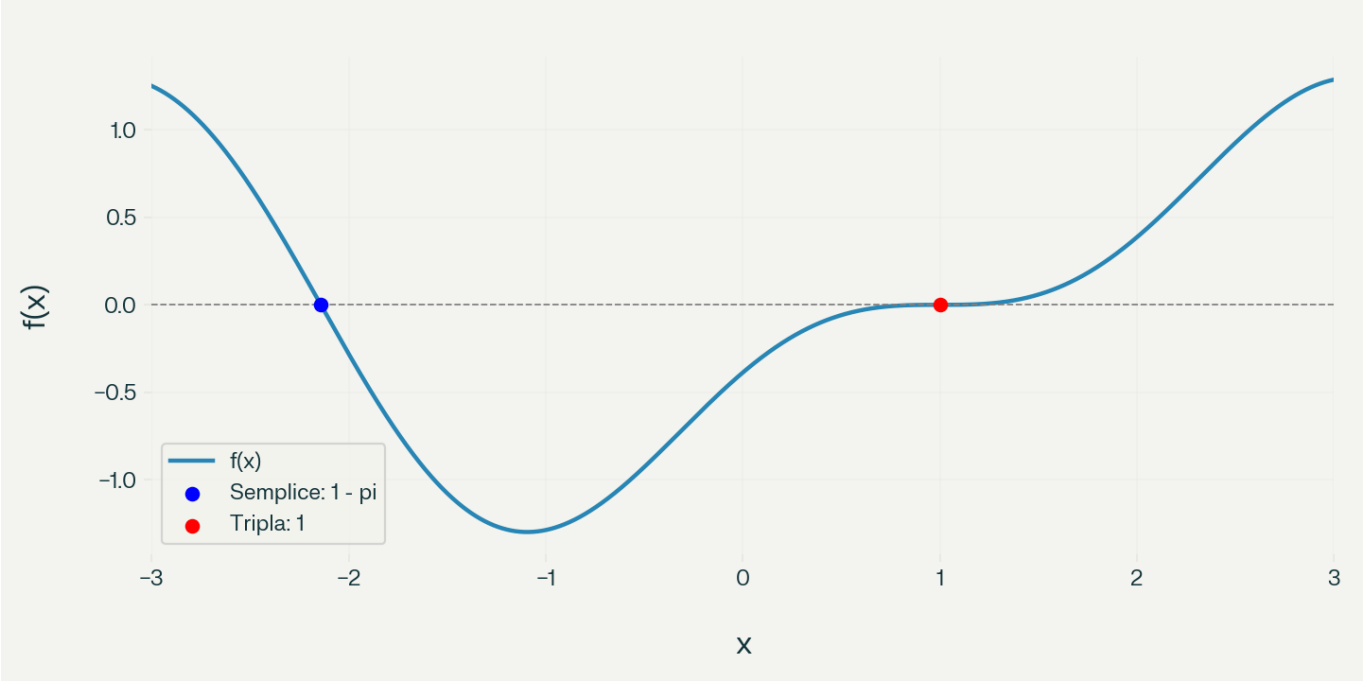


Figura A. Ricostruzione fedele del primo grafico inviato, dalla stessa funzione e dagli stessi punti: il file immagine disponibile per l'impaginazione contiene il secondo screenshot. Questa figura non e una cattura dell'interfaccia MATLAB.

Come leggere la figura A

Punto blu: alpha_s = 1-pi, circa -2.14159. La curva attraversa l'asse con pendenza non nulla: df(alpha_s) = -2. E una radice semplice.

Punto rosso: alpha_m = 1. La curva attraversa ancora l'asse, ma si appiattisce. Vicino a u = 0, sin(u) e circa u e 1-cos(u) e circa u^2/2: dunque f(x) e circa (x-1)^3/2. La radice e tripla. Una radice multipla di molteplicita dispari puo attraversare l'asse: non deve necessariamente limitarsi a toccarlo.

test_multiple - i tuoi grafici degli incrementi

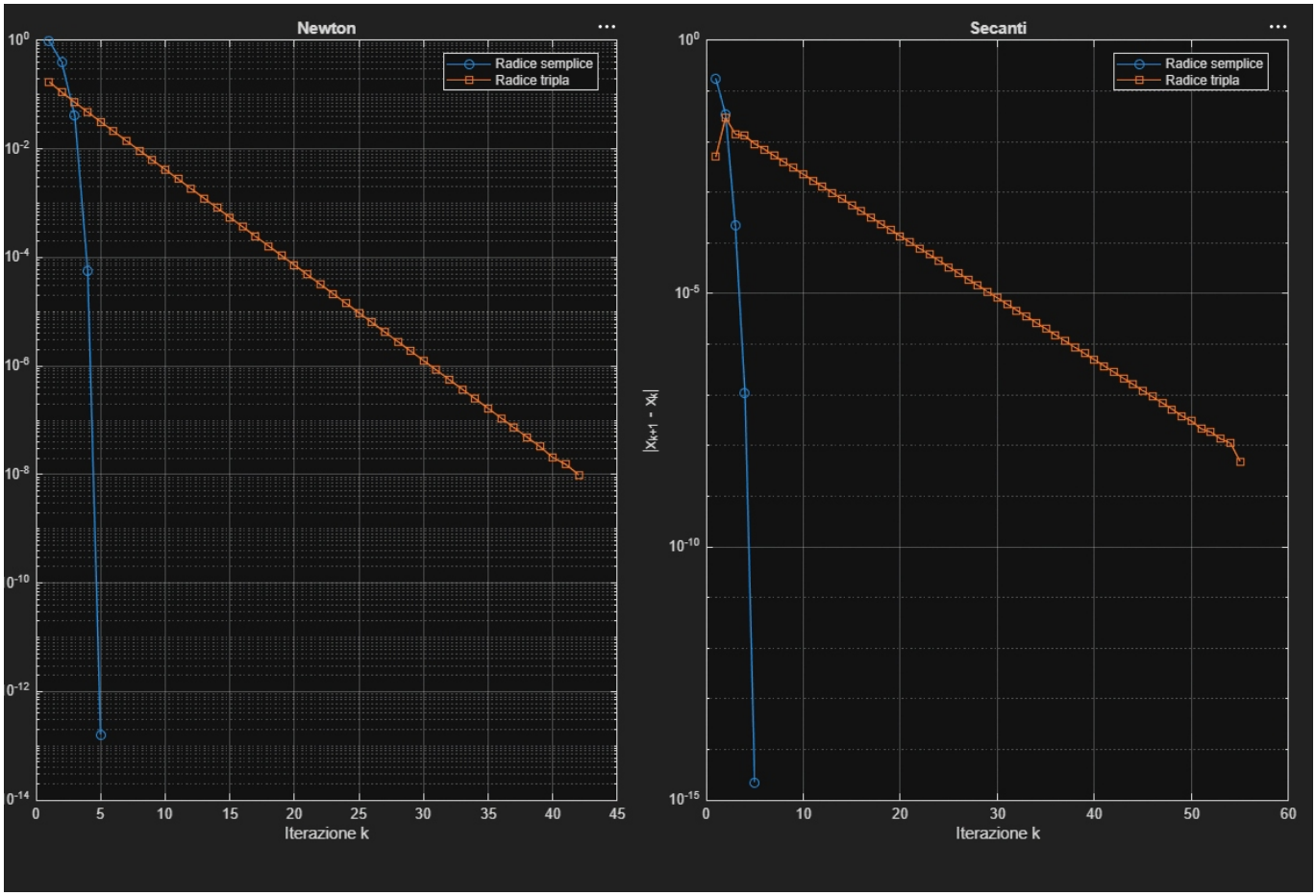


Figura B. Il secondo screenshot che hai inviato, ritagliato per rendere piu leggibili i due pannelli. Asse orizzontale: iterazioni. Asse verticale logaritmico: incremento abs(xnew-xold), non errore rispetto alla radice.

Pannello sinistro: Newton

La curva blu, per la radice semplice, precipita in poche iterazioni, fino a circa 1e-13 nell'ultimo punto visibile. La curva arancione, per la radice tripla, scende quasi lungo una retta e raggiunge circa 1e-8 attorno alla quarantaduesima iterazione. Sono letture approssimative del grafico, non conteggi ricavati dai vettori numerici.

Il comportamento e coerente con una convergenza molto piu rapida sulla radice semplice e lineare su quella tripla. Per Newton su una radice di molteplicita 3 il fattore asintotico dell'errore tende a 2/3: si conserva una frazione dell'errore a ogni passo, invece di ridurlo quadraticamente.

Pannello destro: secanti

La curva blu scende rapidamente e arriva circa a 1e-15 intorno al quinto punto visibile. Quella arancione mostra un breve transitorio iniziale, poi una discesa quasi rettilinea fino a circa 1e-8 verso la cinquantacinquesima iterazione. Anche qui la radice multipla richiede molti piu passi.

I due pannelli NON hanno la stessa scala verticale: quello di Newton arriva circa a 1e-14, quello delle secanti circa a 1e-15, con tacche principali diverse. Non confrontare direttamente l'inclinazione in pixel. Confronta valori, iterazioni e forma del decadimento.

test_multiple - teoria e cautele

1. Cosa suggeriscono le curve

In un grafico semilogy, una riduzione geometrica di un fattore costante appare come una retta. Per questo le curve arancioni sono coerenti con convergenza lineare. Vicino a radici semplici Newton e almeno quadratico nelle usuali ipotesi locali; le secanti hanno tipicamente ordine circa 1.618. L'avvio e gli ultimi passi in precisione finita possono discostarsi dal regime teorico.

Il grafico mostra incrementi: non dimostra da solo l'ordine dell'errore esatto. Per stimarlo direttamente occorre memorizzare le iterate e calcolare abs(x_k-alpha), usando una radice nota.

2. Residuo piccolo su una radice piatta

Vicino a 1, f(x) e circa (x-1)^3/2. Un errore di posizione di 1e-4 produce quindi un residuo dell'ordine di 5e-13. Non attribuire al residuo le stesse cifre di accuratezza della posizione.

3. Zero numerico e cancellazione

La formula sin(u)-0.5*sin(2u) sottrae termini quasi uguali vicino a 0. Gli arrotondamenti possono dominare, dando un residuo numericamente nullo prima di raggiungere esattamente la radice. Anche df usa una differenza delicata. Il codice si ferma sullo spostamento, non con una dimostrazione della distanza esatta dalla radice.

Un incremento zero non e rappresentabile sull'asse logaritmico: puo mancare un punto finale pur presente in ERR. Non confondere l'ultimo punto visibile con il numero esatto di iterazioni: stampa k e il vettore per verificarli.

4. I quattro test e cosa confrontare

Newton semplice: x0 = -1.5, riferimento 1-pi. Newton tripla: x0 = 0.5, riferimento 1. Secanti semplice: x0 = -1.5, x1 = -2. Secanti tripla: x0 = 0.5, x1 = 1.1. Tutti con tol = 1e-8 e kmax = 60.

Per ciascuno controlla z, k, residuo, incremento finale ed errore rispetto al riferimento. Un conteggio piu piccolo in uno specifico test non dimostra superiorita generale: cambiano inizializzazioni, valutazioni di f e uso della derivata.

5. Rendere i pannelli confrontabili

```
% Per entrambi i pannelli, dopo aver disegnato:
xlim([0 60]);
ylim([1e-15 1]);
% Mostra la soglia sugli incrementi:
yline(tol,'k--','Tolleranza','HandleVisibility','off');
```

test_multiple - script da conservare

```
clear; clc;
f = @(x) sin(x-1)-0.5*sin(2*(x-1));
df = @(x) cos(x-1)-cos(2*(x-1));
tol = 1e-8; kmax = 60;
[z_ns,k_ns,res_ns,ERR_ns] = ...
    newton(f,df,-1.5,tol,kmax);
[z_nm,k_nm,res_nm,ERR_nm] = ...
    newton(f,df,0.5,tol,kmax);
[z_ss,k_ss,res_ss,ERR_ss] = ...
    secanti(f,-1.5,-2,tol,kmax);
[z_sm,k_sm,res_sm,ERR_sm] = ...
    secanti(f,0.5,1.1,tol,kmax);

nomi = {'Newton semplice'; 'Newton tripla'; ...
        'Secanti semplice'; 'Secanti tripla'};
z = [z_ns;z_nm;z_ss;z_sm];
k = [k_ns;k_nm;k_ss;k_sm];
res = [res_ns;res_nm;res_ss;res_sm];
alpha = [1-pi;1;1-pi;1];
for j = 1:4
    fprintf('\n%s: z=%.16f, k=%d\n',nomi{j},z(j),k(j));
    fprintf('Residuo=%.3e, errore=%.3e\n', ...
            res(j),abs(z(j)-alpha(j)));
end

figure(2); clf; tiledlayout(1,2);
nexttile;
semilogy(1:length(ERR_ns),ERR_ns,'o-'); hold on;
semilogy(1:length(ERR_nm),ERR_nm,'s-');
title('Newton'); xlabel('k'); ylabel('Incremento');
legend('Semplice','Tripla'); grid on;
xlim([0 60]); ylim([1e-15 1]);
nexttile;
semilogy(1:length(ERR_ss),ERR_ss,'o-'); hold on;
semilogy(1:length(ERR_sm),ERR_sm,'s-');
title('Secanti'); xlabel('k'); ylabel('Incremento');
legend('Semplice','Tripla'); grid on;
xlim([0 60]); ylim([1e-15 1]);
```

Fonti e materiali

Script MATLAB e risultati condivisi durante lo studio; grafici inviati dall'utente. Riferimenti teorici: University of Waterloo, Topic 10.1 Bisection e Topic 10.4 Secant Method; UBC, The Newton-Raphson Method; University of Colorado, Solving Scalar Nonlinear Equations; UPC, A note on the convergence of the secant method for simple and multiple roots; MathWorks, documentazione semilogy, while e operatori logici.